Clase 07 - Subrutinas

IIC2343 - Arquitectura de Computadores

Profesor:
- Felipe Valenzuela González
Correo:
frvalenzuela@alumni.uc.cl

Resumen de la clase pasada

Salto Incondicional: Instrucciones (ejemplo)

- La limitante es el hecho de **no iterar**
- Las instrucciones que nos permiten cambiar la dirección de código las llamaremos **saltos**
- En particular en el ejemplo tenemos un **salto incondicional**

Dirección	Instrucción	Operandos
0x00	MOV	A,0
0x01	MOV	B,1
0x02	ADD	A,B
0x03	ADD	B,A
0x04	JMP	0x02

Imagen Referencia: https://github.com/IIC2343/Syllabus-anteriores/blob/main/Apuntes%20(viejos)/5%20-%20Saltos%20y%20Subrutinas.pdf

Salto Incondicional: Label

- Es un **indicador** que se puede agregar en una línea del código assembly para referirse a la **dirección de memoria** asociada a esa línea

Dirección	Label	Instrucción	Operandos
0x00		MOV	A,0
0x01		MOV	B,1
0x02	start:	ADD	A,B
0x03		ADD	B,A
0x04		JMP	start

Imagen Referencia: https://github.com/IIC2343/Syllabus-anteriores/blob/main/Apuntes%20(viejos)/5%20-%20Saltos%20y%20Subrutinas.pdf

Salto Incondicional: Implementación

- La respuesta es modificar el resgistro del **Program Counter**
- Esta conexión va desde el literal que viene de la **Instruction Memory**
- Agregamos una nueva señal de control **Lpc** (Load PC)

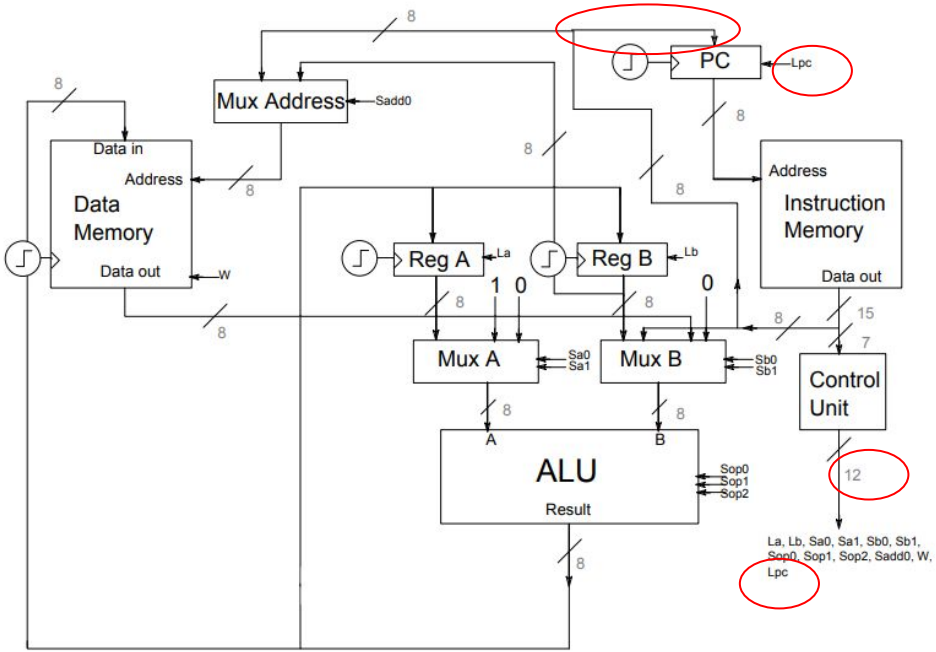

Imagen Referencia: https://github.com/IIC2343/Syllabus-anteriores/blob/main/Apuntes%20(viejos)/5%20-%20Saltos%20y%20Subrutinas.pdf

Salto condicional: Implementación

- Dado que el salto se realiza con las condiciones ocurridas en la operación anterior
- Se necesita un nuevo registro **Status**
- Su salida se conecta la Unidad de Control

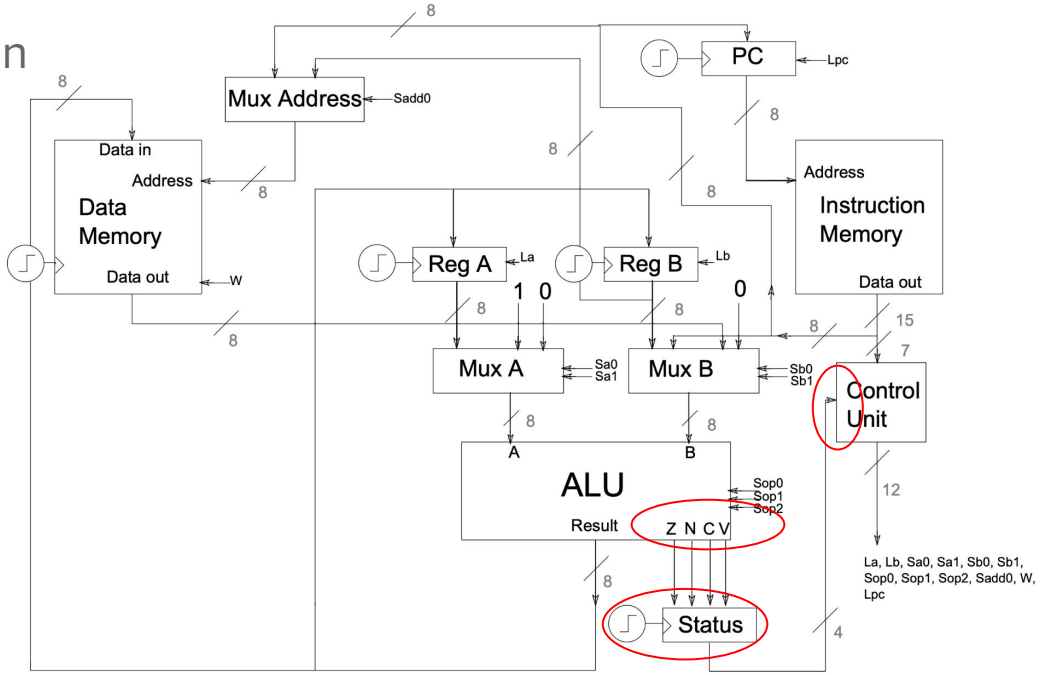

Imagen Referencia: https://github.com/IIC2343/Syllabus-anteriores/blob/main/Apuntes%20(viejos)/5%20-%20Saltos%20y%20Subrutinas.pdf

Salto Resumen

Instrucción	Operandos	Operación	Condiciones	Ejemplo de uso
CMP	A,B	A-B		
	A,Lit	A-Lit		CMP A,0
JMP	Dir	PC = Dir		JMP end
JEQ	Dir	PC = Dir	Z=1	JEQ label
JNE	Dir	PC = Dir	Z=0	JNE label
JGT	Dir	PC = Dir	N=0 y Z=0	JGT label
JLT	Dir	PC = Dir	N=1	JLT label
JGE	Dir	PC = Dir	N=0	JGE label
JLE	Dir	PC = Dir	Z=1 o N=1	JLE label
JCR	Dir	PC = Dir	C=1	JCR label
JOV	Dir	PC = Dir	V=1	JOV label

Imagen Referencia: https://github.com/IIC2343/Syllabus-anteriores/blob/main/Apuntes%20(viejos)/5%20-%20Saltos%20y%20Subrutinas.pdf

Introducción del curso:

- Un computador lo definimos como una **máquina programable que ejecuta programas.** ✓
- Para programar necesitamos:
    - Datos: números (enteros, reales) , texto, imágenes, etc ✓
    - Operaciones: suma, resta, multiplicación, división, etc ✓
    - Variables: simples, arreglos ✓
    - Control de flujo: comparaciones, manejo de ciclos ✓
- Ahora podemos volver a revisar un que es un programa...

Introducción: ¿Qué es un programa?

```python
def encontrar_maximo(arreglo):
    largo_maximo = len(arreglo)

    maximo = arreglo[0]
    i = 0

    while i < largo_maximo:
        if arreglo[i] > maximo:
            maximo = arreglo[i]
        i += 1

    return maximo
```

Introducción: ¿Qué es un programa?

```
def encontrar_maximo(arreglo):
    largo_maximo = len(arreglo)

    maximo = arreglo[0]
    i = 0

    while i < largo_maximo:
        if arreglo[i] > maximo:
            maximo = arreglo[i]
        i += 1

    return maximo
```

NOS FALTA!!

SUBRUTINAS

¿Dudas?

Subrutina: Ejemplo

- Con las herramientas actuales podemos escribir este código en assembly

```
def encontrar_maximo(arreglo):
    largo_maximo = len(arreglo)

    maximo = arreglo[0]
    i = 0

    while i < largo_maximo:
        if arreglo[i] > maximo:
            maximo = arreglo[i]
        i += 1

    return maximo
```

Subrutina: Ejemplo

- Definimos las variables:

```
DATA:
arr 1
    5
    3
len 3
max -1
i   0
bool 0
```

```python
def encontrar_maximo(arreglo):
    largo_maximo = len(arreglo)

    maximo = arreglo[0]
    i = 0

    while i < largo_maximo:
        if arreglo[i] > maximo:
            maximo = arreglo[i]
        i += 1

    return maximo
```

Subrutina: Ejemplo

- Inicializar:

```
CODE:
update_max:
MOV A,(i)
MOV (max),A
MOV A, (bool)
CMP A, 1
JEQ increment
```

```python
def encontrar_maximo(arreglo):
    largo_maximo = len(arreglo)

    maximo = arreglo[0]
    i = 0

    while i < largo_maximo:
        if arreglo[i] > maximo:
            maximo = arreglo[i]
        i += 1

    return maximo
```

Subrutina: Ejemplo

- Iterador:

```
while:
MOV A,(i)
MOV B,(len)
CMP A,B
JLT if
JMP end
```

```python
def encontrar_maximo(arreglo):
    largo_maximo = len(arreglo)

    maximo = arreglo[0]
    i = 0

    while i < largo_maximo:
        if arreglo[i] > maximo:
            maximo = arreglo[i]
        i += 1

    return maximo
```

Subrutina: Ejemplo

- Condicional:

```
if:
MOV A, 1
MOV (bool), A
MOV B,(i)
MOV A,(B)
MOV B,(max)
CMP A,B
JGT update_max
```

```
def encontrar_maximo(arreglo):
    largo_maximo = len(arreglo)

    maximo = arreglo[0]
    i = 0

    while i < largo_maximo:
        if arreglo[i] > maximo:
            maximo = arreglo[i]
        i += 1

    return maximo
```

Subrutina: Ejemplo

- Incrementar:

```
increment:
MOV B,(i)
INC B
MOV (i),B
JMP while
```

```python
def encontrar_maximo(arreglo):
    largo_maximo = len(arreglo)

    maximo = arreglo[0]
    i = 0

    while i < largo_maximo:
        if arreglo[i] > maximo:
            maximo = arreglo[i]
        i += 1

    return maximo
```

Subrutina: Ejemplo

- El código final se vería:

```
DATA:
arr 1
     5
     3
len 3
max -1
i   0
bool 0
CODE:
update_max:
MOV A,(i)
MOV (max),A
MOV A, (bool)
CMP A, 1
JEQ increment
```

```
while:
MOV A,(i)
MOV B,(len)
CMP A,B
JLT if
JMP end
if:
MOV A, 1
MOV (bool), A
MOV B,(i)
MOV A,(B)
MOV B,(max)
CMP A,B
JGT update_max
```

```
increment:
MOV B,(i)
INC B
MOV (i),B
JMP while
end:
MOV A,(max)
MOV B,(max)
JMP end
```

¿Dudas?

Subrutina: Ejemplo

- Con las herramientas actuales podemos escribir este código en assembly
- Si necesitamos calcular el máximo de 5 arreglos, significa reescribir en assembly **el mismo código múltiples veces**

```
def encontrar_maximo(arreglo):
    largo_maximo = len(arreglo)

    maximo = arreglo[0]
    i = 0

    while i < largo_maximo:
        if arreglo[i] > maximo:
            maximo = arreglo[i]
        i += 1

    return maximo
```

Subrutina: Requisitos

- Parámetro de Entrada
- Valor de retorno
- Llamada a la subrutina (salto de ida y de vuelta)

```
def encontrar_maximo(arreglo):
    largo_maximo = len(arreglo)

    maximo = arreglo[0]
    i = 0

    while i < largo_maximo:
        if arreglo[i] > maximo:
            maximo = arreglo[i]
        i += 1

    return maximo
```

Subrutina: Parámetro de Entrada y Valor Retorno

- Tenemos dos opciones:
- **Almacenamiento por Registro:** Muy limitado en la arquitectura actual
- **Almacenamiento por Memoria:** Ocupar espacio de la memoria para almacenar. El limitante es más lento pero dado el espacio es el que **utilizaremos**

Subrutina: Llamada y Retorno

- Tenemos el desafío de saber el **valor al que estamos saltamos**
- Necesitamos **Almacenar el valor del program counter**

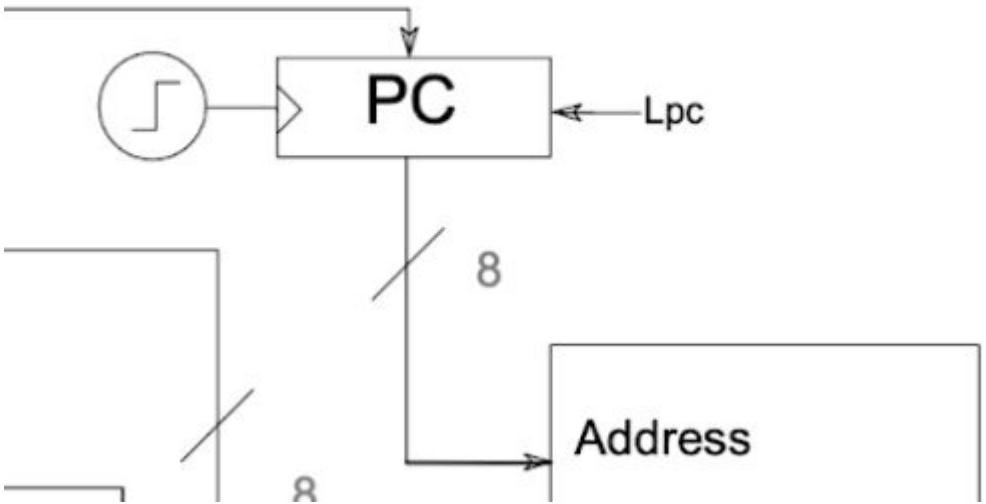

Imagen Referencia: https://github.com/IIC2343/Syllabus-anteriores/blob/main/Apuntes%20(viejos)/5%20-%20Saltos%20y%20Subrutinas.pdf

Subrutina: Implementación

- Agregamos un **Mux DataIn** a la entrada de la Memoria
- Agregamos un **Mux PC** a la entrada del Program Counter
- ¿Cómo nos aseguramos que no choquen las variables?

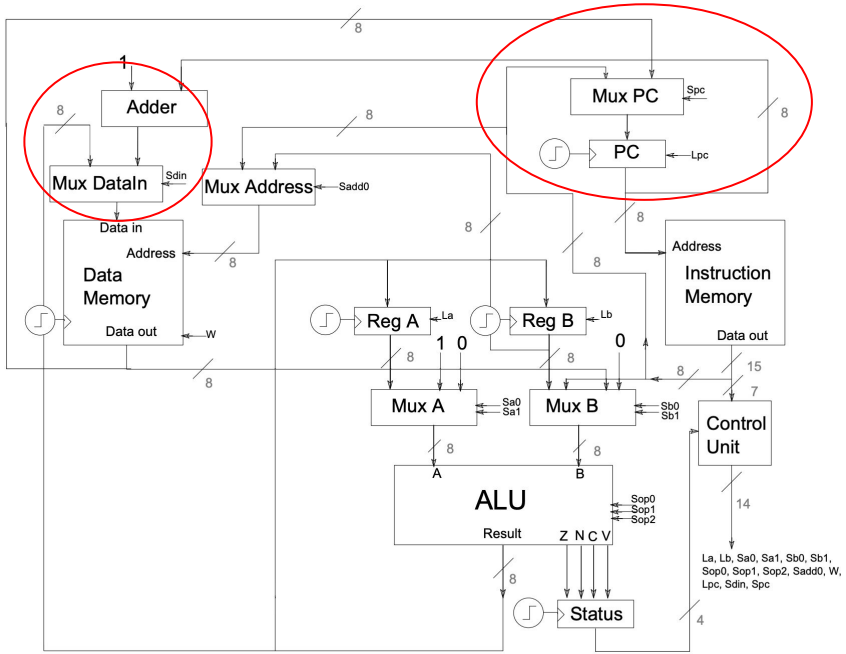

Imagen Referencia: https://github.com/IIC2343/Syllabus-anteriores/blob/main/Apuntes%20(viejos)/5%20-%20Saltos%20y%20Subrutinas.pdf

Subrutina: Implementación - SP

- Agregamos un nuevo registro **SP** a la entrada del Mux Address
- Su valor parte en 255 al ser de 8 bits (el valor más alto en base 10)

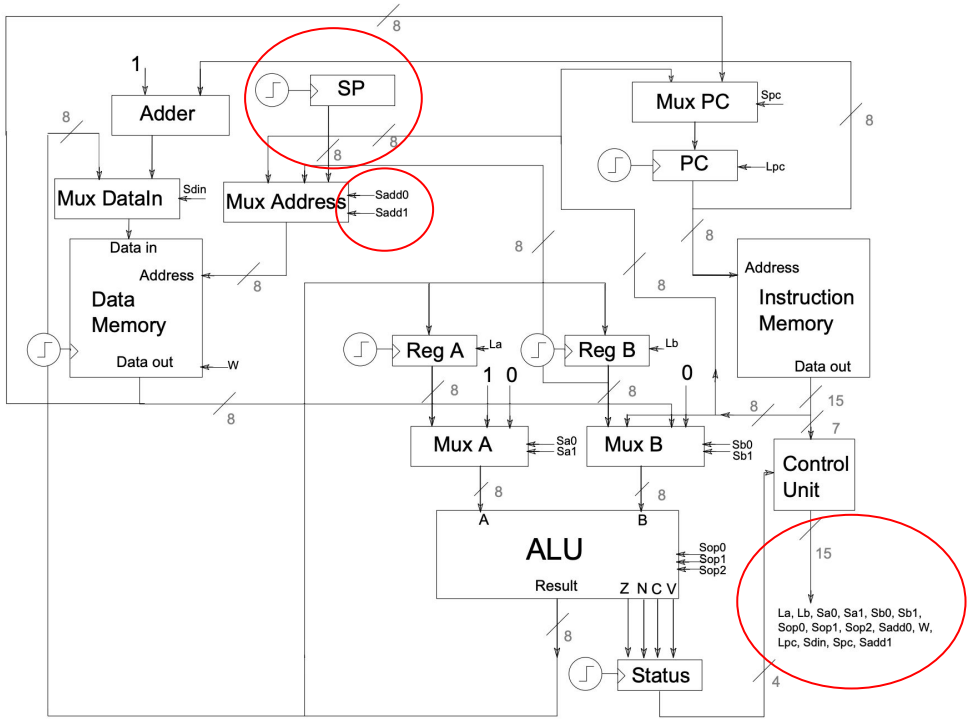

Imagen Referencia: https://github.com/IIC2343/Syllabus-anteriores/blob/main/Apuntes%20(viejos)/5%20-%20Saltos%20y%20Subrutinas.pdf

¿Dudas?

Subrutina: Memoria de Stack

- Utilizaremos el modelo de pila o en inglés Stack
- Si el *stack* choca con las variables definidas se produce un **stackoverflow**

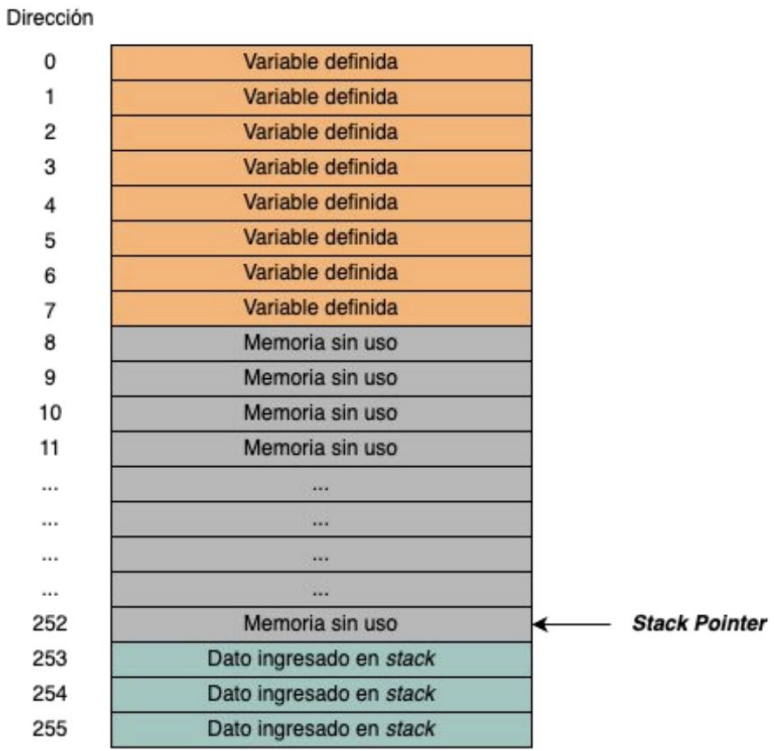

Imagen Referencia:https://github.com/IIC2343/Syllabus-2023-2/blob/main/clases/s1-s3/Clase%205%20-%20Saltos%20y%20Subrutinas.pdf

Subrutina: Assembly - CALL

- CALL label
- Almacena el valor PC+1 en la dirección de memoria SP
- Reduce en una unidad el contador SP
- Se realiza en **un ciclo de clock**

```
...
...
CALL subrutina1
...
```

S_{DIn} = PC+1; S_{Add} = SP; W = 1;

Dec_{SP} = 1; S_{PC} = LIT; L_{PC} = 1

Imagen Referencia:https://github.com/IIC2343/Syllabus-2023-2/blob/main/clases/s1-s3/Clase%205%20-%20Saltos%20y%20Subrutinas.pdf

Subrutina: Assembly - RET

- RET
- Aumenta en una unidad el contador SP
- Extrae de memoria el valor almacenado en la dirección SP
- Carga dicho valor en PC
- Se realiza en **dos ciclo de clock**

```
CALL subrutina2
...
RET
...
```

Ciclo 1: $Inc_{SP} = 1$

Ciclo 2: $S_{Add} = SP;$

$S_{PC} = DOUT; L_{PC} = 1$

Imagen Referencia:https://github.com/IIC2343/Syllabus-2023-2/blob/main/clases/s1-s3/Clase%205%20-%20Saltos%20y%20Subrutinas.pdf

Subrutina: Assembly - PUSH y POP

- **PUSH Reg**
- Almacena el valor de un registro en la dirección de memoria SP (toma un ciclo)
- **POP Reg**
- Aumenta en una unidad el contador SP y almacena en el registro Reg el dato almacenado en la dirección SP (toma dos ciclos)

```
MOV  A,5
MOV  B,3
PUSH A
PUSH B
CALL subrutina
POP  B
POP  A
```

Imagen Referencia: https://github.com/IIC2343/Syllabus-anteriores/blob/main/Apuntes%20(viejos)/5%20-%20Saltos%20y%20Subrutinas.pdf

Subrutina: Assembly - Resumen

a) Escriba en el assembly del computador básico un programa que calcule la división entre 2 números enteros, a y b, mediante la llamada a la subrutina $q = div(a, b)$, que también debe ser implementada. Cualquier detalle de implementación o aspecto que se asuma debe quedar claramente explicado. **(2 ptos.)**

Solución: Una implementación en Java de la división entera es la siguiente:

```
public byte div(byte a, byte b){
    byte res = 0;
    a = a-b;
    while (a >= 0)
    {
        res = res + 1;
        a = a-b;
    }
    return res;}
```

Imagen Referencia: https://github.com/IIC2343/Syllabus-anteriores/blob/main/Apuntes%20(viejos)/5%20-%20Saltos%20y%20Subrutinas.pdf

Subrutina: Ejercicio I2 - 2012

a) Escriba en el assembly del computador básico un programa que calcule la división entre 2 números enteros, a y b, mediante la llamada a la subrutina $q = div(a, b)$, que también debe ser implementada. Cualquier detalle de implementación o aspecto que se asuma debe quedar claramente explicado. **(2 ptos.)**

Solución: Una implementación en Java de la división entera es la siguiente:

```
public byte div(byte a, byte b){
    byte res = 0;
    a = a-b;
    while (a >= 0)
    {
        res = res + 1;
        a = a-b;
    }
    return res;}
```

Imagen Referencia: https://github.com/IIC2343/Syllabus-anteriores/blob/main/Apuntes%20(viejos)/5%20-%20Saltos%20y%20Subrutinas.pdf

Subrutina: Ejercicio I2 - 2012

a) Escriba en el assembly del computador básico un programa que calcule la división entre 2 números enteros, a y b, mediante la llamada a la subrutina $q = div(a, b)$, que también debe ser implementada. Cualquier detalle de implementación o aspecto que se asuma debe quedar claramente explicado. **(2 ptos.)**

Solución: Una implementación en Java de la división entera es la siguiente:

```
public byte div(byte a, byte b){
    byte res = 0;
    a = a-b;
    while (a >= 0)
    {
        res = res + 1;
        a = a-b;
    }
    return res;}
```

Imagen Referencia: https://github.com/IIC2343/Syllabus-anteriores/blob/main/Apuntes%20(viejos)/5%20-%20Saltos%20y%20Subrutinas.pdf

Subrutina: Ejercicio I1 - 2024-2

(a) (1.5 ptos.) Indique los valores de los registros `A` y `B` al finalizar la ejecución del código **(a)**.

(b) (1.5 ptos.) Indique los valores de los registros `A` y `B` al finalizar la ejecución del código **(b)**.

(c) (3 ptos.) El código **(c)** *debería* computar la multiplicación entre las variables `X` e `Y`, pero cuenta con errores que generan un resultado erróneo. Indique el valor de los registros `A`, `B` y la variable `res` al finalizar la ejecución del código. Luego, señale dónde se encuentra el o los errores y cómo se resuelven.

Subrutina: Ejercicio I1 - 2024-2

(a)

```
DATA:
res     0

CODE:
JMP _start

func_1:
SHR A, A
JCR func_2
CMP A, 0
JNE func_1
RET

func_2:
NOT B, A
MOV (B), 14
INC (res)
JMP func_1

_start:
MOV A, 6
CALL func_1
MOV B, (res)
```

(b)

```
DATA:
var   10

CODE:
JMP _start

func_1:
MOV B, (var)
NOT A, A
ADD A, 1
ADD B, A
RET

_start:
MOV A, 3
CALL func_1

end:
MOV A, (255)
```

(c)

```
DATA:
res     0
X       3
Y       4
iter    0

CODE:
MOV A, (Y)
PUSH A
CALL mult
MOV A, (res)
JMP end

mult:
POP B
sumar:
MOV A, (res)
ADD A, (X)
MOV (res), A
INC (iter)
MOV A, (iter)
CMP A, B
JNE sumar
RET

end:
```

Subrutina: Ejercicio I1 - 2024-2

Pregunta 4: Computador básico (6 ptos.)

Asuma que se encuentra trabajando como programador(a) en la oficina del famoso juego desarrollado completamente en Assembly: *Rollercoaster Tycoon*. Luego de unas horas, se da cuenta que la letra `T` del teclado de su computador ha dejado de funcionar, la que es de suma importancia al momento de escribir subrutinas por el uso de la instrucción `RET`. Adicionalmente, no cuenta con la opción de copiar y pegar texto, por lo que deberá desistir de su uso. Por el motivo anterior, deberá diseñar tres nuevas instrucciones que le permitan simular parte del comportamiento de la instrucción `RET`.

1. `MOV B, SP`: Almacena el valor del registro `SP` en el registro `B`.

¿Dudas?

Introducción del curso:

- Un computador lo definimos como una **máquina programable que ejecuta programas.** ✓
- Para programar necesitamos:
    - Datos: números (enteros, reales) , texto, imágenes, etc ✓
    - Operaciones: suma, resta, multiplicación, división, etc ✓
    - Variables: simples, arreglos ✓
    - Control de flujo: comparaciones, manejo de ciclos ✓
- Y nos permite re-utilizar trozos de código ✓

Introducción del curso:

- Un computad[illegible]s co[illegible] **progra[illegible]e que ejecuta programas** ✅
- Para prog[illegible]
  - [illegible]
  - [illegible]
  - [illegible]
  - [illegible]
- Y nos pe[illegible]te

Imagen Referencia: https://aceattorney.fandom.com/wiki/Objection!

Introducción del curso:

- Un computador lo definimos como una **máquina programable que ejecuta programas.** ✓
- Para programar necesitamos: ?
    - Datos: números (enteros, reales) , texto, imágenes, etc ? ?
    - Operaciones: suma, resta, multiplicación, división, etc ✓
    - Variables: simples, arreglos ✓
    - Control de flujo: comparaciones, manejo de ciclos ✓
- Para poder tener números distinto a enteros necesitamos ¡**punto flotante**!

Clase 07 - Subrutinas

IIC2343 - Arquitectura de Computadores

Profesor:
- Felipe Valenzuela González
Correo:
frvalenzuela@alumni.uc.cl